SAP ABAP LAB 001

Manual operativo de desarrollo agentic ABAP

Hermes Desktop · Claude Code · Claude Desktop · ARC-1

Sistema: SAP S/4HANA 2025 on-premise · Cliente 100

Documento generado desde el workspace versionable. No contiene credenciales.

1. Propósito

Este workspace permite trabajar de forma consistente desde Hermes Desktop, Claude Code y Claude Desktop contra el sistema SAP S/4HANA 2025 del laboratorio. Combina reglas portables, ARC-1, skills ABAP, control por Git y un ciclo de aprendizaje continuo basado en evidencia.

2. Arquitectura

- Hermes Desktop carga las reglas del proyecto y sus skills SAP locales; ARC-1 aparece como MCP nativo.
- Claude Code usa `CLAUDE.md`, los plugins `clean-abap`, `abap-cloud-rap` y `arc-1`.
- Claude Desktop usa el MCP `sap-arc1-s4h-100` mediante un launcher local sin credenciales en su JSON. Los plugins de Claude Code no existen en Claude Desktop.
- ARC-1 standalone conecta directamente por HTTPS/ADT al sistema SAP.
- Las credenciales residen únicamente en el almacén privado local.

3. Estado del sistema

- SAP S/4HANA 2025 on-premise.
- ABAP Platform 2025 / SAP_BASIS 816.
- Cliente 100, SID S4H.
- HANA, RAP/CDS, AMDP, UI5/Fiori, CTS, gCTS, UI5 Repository Deploy y FLP disponibles.
- abapGit no disponible: endpoint ICF no activo.
- Búsqueda full-text ADT no disponible: servicio ICF no activo.
- Autenticación verificada con una lectura `SYSTEM` no destructiva.

4. Riesgo TLS pendiente

La configuración usa temporalmente `SAP_INSECURE=true` porque el certificado del endpoint SAP necesita renovación. Esto desactiva la validación TLS solo para el proceso ARC-1. Basis debe renovar el certificado; después se debe eliminar la excepción en Hermes y en el launcher de Claude Desktop.

5. Inicio con Hermes Desktop

1. Abrir el proyecto `SAP ABAP LAB 001` en Hermes Desktop.
2. Confirmar que la carpeta activa es este repositorio.
3. Pedir a Hermes que lea `AGENTS.md` y `config/system.yaml`.
4. Verificar `sap_arc1_s4h_100` con `hermes mcp list` o pedir una lectura inocua del sistema.
5. Pedir la tarea en español, indicando paquete/objeto cuando se conozcan.
6. Hermes cargará la skill SAP aplicable antes de actuar.

Skills Hermes disponibles:

- `clean-abap-review`

- `clean-abap-refactor`
- `abap-cloud-rap-design`
- `abap-atc-remediation`
- `abap-clean-core-check`
- `sap-abap-agentic-development`

Ejemplos:

- "Revisa la clase ZCL_* con Clean ABAP; no escribas cambios."
- "Diseña un BO RAP para ..., primero presenta objetos y dependencias."
- "Remedia estos hallazgos ATC, activa y vuelve a ejecutar ATC."

6. Inicio con Claude Code

1. Abrir una terminal en la raíz del repositorio y ejecutar `claude`.
2. Claude Code carga `CLAUDE.md`, que remite a `AGENTS.md`.
3. Verificar plugins con `claude plugin list`.
4. Usar los comandos:

- `/clean-abap:review`
- `/clean-abap:refactor`
- `/abap-cloud-rap:rap-bo-design`
- `/abap-cloud-rap:atc-remediation`
- `/abap-cloud-rap:clean-core-check`

5. Para generadores RAP de larga duración iniciar Claude Code con un timeout MCP suficiente.

7. Inicio con Claude Desktop

1. Cerrar y abrir completamente Claude Desktop después de cambios MCP.
2. En Settings → Developer → Local MCP servers, confirmar `sap-arc1-s4h-100` como running.
3. Crear un Project de Claude Desktop para este laboratorio.
4. Añadir como conocimiento/instrucciones:

- `docs/CLAUDE-DESKTOP-PROJECT-INSTRUCTIONS.md`
- `AGENTS.md`
- `config/system.yaml`
- los archivos de `learning/` que sean relevantes.

5. Pedir una lectura `SYSTEM` o búsqueda inocua antes de cualquier cambio.

Nota: Claude Desktop puede usar ARC-1, pero no carga los plugins/slash commands de Claude Code.

8. Herramientas ARC-1

Con la política completa, ARC-1 1.1.0 publica:

- `SAPContext`: intención, dependencias e impacto.
- `SAPSearch`: objetos, TADIR y búsqueda.
- `SAPRead`: fuente, DDIC, sistema, paquetes, borradores y lecturas de tabla controladas.
- `SAPNavigate`: definiciones, where-used y jerarquías.
- `SAPLint`: lint, formato y correcciones locales.
- `SAPDiagnose`: sintaxis, ABAP Unit, ATC y diagnóstico.
- `SAPWrite`: crear, actualizar, borrar y editar objetos.
- `SAPActivate`: activar objetos y publicar/despublicar SRVB.
- `SAPManage`: capacidades, paquetes, FLP y estado de APIs.
- `SAPTransport`: CTS, creación/liberación/borrado/reasignación.
- `SAPGit`: gCTS/abapGit según disponibilidad.
- `SAPQuery`: SELECT Open SQL de solo lectura.

La lista expuesta por la sesión en vivo siempre prevalece sobre este manual.

9. Flujo para crear o modificar ABAP

1. Contexto: `SAPContext`.
2. Descubrimiento: `SAPSearch` y `SAPRead`.
3. Impacto: `SAPNavigate`/where-used, dependencias y contratos API.
4. Diseño: objetos, paquete, transporte y reversión.
5. Diff mínimo: método/unidad antes que objeto completo.
6. Prevalidación: `SAPLint` y `SAPDiagnose syntax`.
7. Escritura: `SAPWrite`.
8. Activación: `SAPActivate`; batch para dos o más objetos.
9. Pruebas: ABAP Unit y ATC cuando correspondan.
10. Confirmación: lectura activa, `INACTIVE_OBJECTS` limpio y documentación.

Una respuesta exitosa de `SAPWrite` significa que la escritura pasó las gates configuradas; no significa que compile. La activación es el gate definitivo.

10. Paquetes y transportes

- `$TMP`: experimentos desechables.
- `Z*`/`Y*`: desarrollos durables.
- No inventar transporte, capa o target; consultarlos con ARC-1 o pedirlos.
- Para objetos transportables, registrar transporte y estrategia de reversión.
- La liberación o eliminación de transportes requiere confirmación explícita inmediata.

11. SQL y datos

El acceso de consulta está habilitado técnicamente. Usarlo solo con propósito explícito:

- SELECT de solo lectura;
- columnas mínimas;
- filtros restrictivos;
- `maxRows` pequeño;
- nunca exportaciones masivas;
- especial cuidado con HR, finanzas, clientes y datos personales.

Los resultados enviados a un agente pueden formar parte del contexto enviado al proveedor del modelo.

12. Aprendizaje continuo

Después de cada trabajo verificado:

- `learning/lessons.md`: lecciones reutilizables con evidencia.
- `learning/decisions.md`: decisiones de arquitectura.
- `learning/patterns.md`: patrones ABAP aprobados.
- `learning/backlog.md`: deuda y mejoras pendientes.

Reglas de automejora:

- no guardar secretos;
- no convertir hipótesis en conocimiento;
- reemplazar notas obsoletas;
- evitar duplicados;
- mantener cada cambio revisable con Git.

13. Validación local

Ejecutar desde la raíz:

```
python scripts/verify_workspace.py
```

El validador revisa archivos obligatorios, YAML, patrones de secretos, MCP de Hermes, plugins de Claude Code y la entrada secret-free de Claude Desktop.

14. Operación segura aun con acceso completo

El acceso técnico está pensado para desarrollo completo, pero estas acciones necesitan confirmación concreta antes de ejecutarse:

- borrar objetos o paquetes;
- modificar SAP estándar;
- liberar/borrar/reasignar transportes;

- publicar o despublicar servicios;
- operaciones mutantes de gCTS/abapGit;
- cualquier acción SQL o programa que cambie datos de negocio.

15. Recuperación y reversión

Antes de cambios importantes:

- leer versión activa y revisiones;
- guardar diff y objetos afectados;
- comprobar transporte;
- preferir cambios pequeños;
- activar por batch cuando haya dependencias;
- si falla, revisar versión inactiva, errores de activación y orden de dependencias antes de reintentar.

16. Mantenimiento

- Actualizar ARC-1 y plugins de forma controlada; ejecutar smoke tests después.
- Revisar el catálogo en vivo porque herramientas y schemas evolucionan.
- Renovar el certificado y eliminar `SAP_INSECURE=true`.
- Revisar `learning/` y el manual tras cambios de arquitectura.